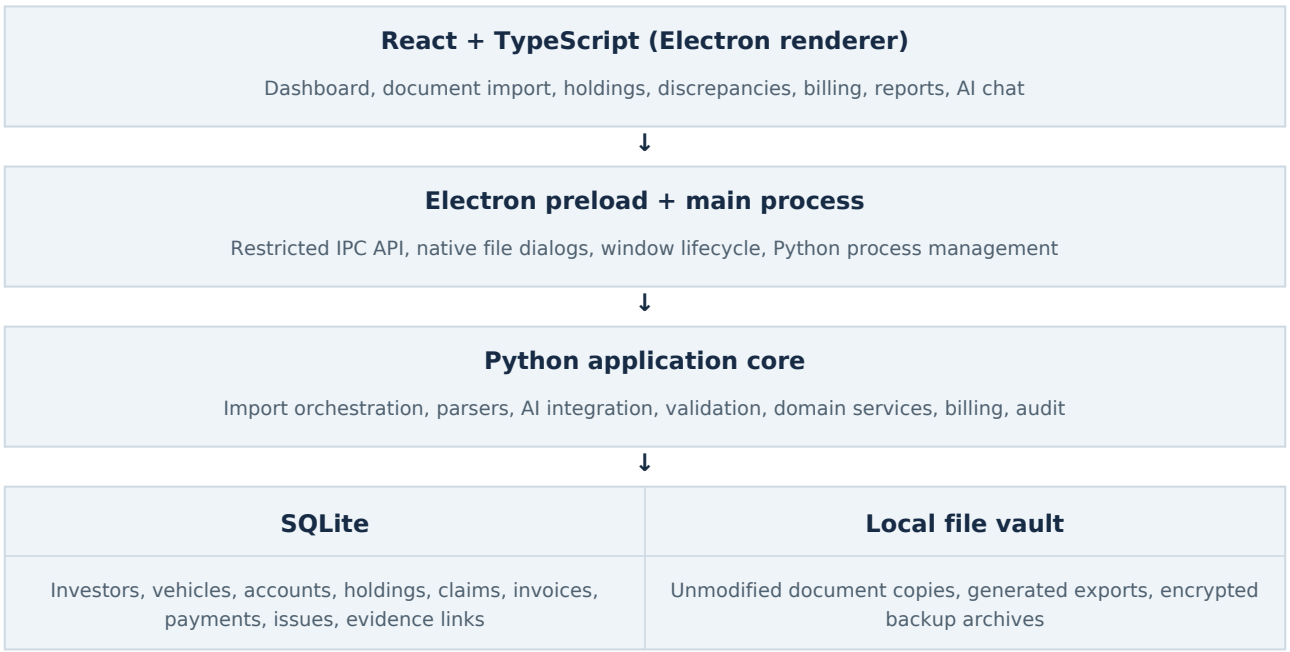
Group Claims Management System

Implementation architecture | Electron + React + Python | macOS / Windows | Single-user desktop

1. System architecture



External service: Python communicates with an authorized cloud AI provider over HTTPS for document interpretation and chat. No local AI model. No application-hosted document server.

2. Component responsibilities

Component	Responsibility
Electron main + preload	Desktop shell; expose allowlisted commands to React via secure IPC; supervise packaged Python process.
Python parsers	Read Excel/CSV values and PDF text, preserving sheet/row/page source references.
Cloud AI integration	Interpret unstructured text and irregular layouts; return evidence-linked structured proposals; assist chat.
Validation + reconciliation	Check required fields, ISINs, totals, duplicate imports, dated positions, entity conflicts and material changes.
Domain services	Maintain distinct investors, vehicles, legal entities, accounts, positions, claims/rights and invoice recipients.
Billing engine	Apply configured fee terms and pro-rata rules using exact decimal arithmetic; create invoice versions and adjustments.
SQLite + file vault	Persist structured records and stable references to immutable source-file copies.

Core processing and financial controls

3. Import-to-billing flow

Step	System action
1. Import	User selects files; app copies originals to a managed local folder; records file hash and provenance.
2. Parse	Python extracts cells/text and source locations. Standard layouts can bypass AI.
3. Interpret	Cloud AI interprets ambiguous headings, legal language and relationships where necessary; authorized content only.
4. Validate	Deterministic checks reconcile facts with prior dated records. Identical files do not create duplicate holdings.
5. Handle issues	Ordinary valid records auto-process. Material unresolved discrepancies enter an issues queue; affected billing is blocked.
6. Persist	SQLite stores normalized records, evidence references, timestamps and change history.
7. Calculate	Billing engine selects eligible holdings/claimants, applies agreed fee rules, and produces traceable amounts.
8. Deliver	User reviews significant issues and outputs; issues invoices, records payments and exports reports.

4. Domain data model

Investor / legal entity → investment vehicle / subfund → account → dated holding position → security (ISIN). Claim-right records separately identify asserted claimant, scope, relevant dates and evidence. Billing agreements link to an invoice recipient and fee calculation rules.

Reason: Security holder, legal claimant and invoice recipient may be different parties. Holdings and claim-right transfers must not be inferred from one another. Legal interpretations that remain uncertain are flagged.

5. Billing engine

Input / operation	Control
Inputs	Dated eligible positions/claims, approved fee agreement terms, currency/tax policy, billing period.
Allocation	Example rule: entity eligible base ÷ total eligible base × fee pool. Actual rule is contract-specific.
Precision	Decimal arithmetic and a documented rounding policy; every allocated fee reconciles to the fee pool.
Invoice lifecycle	Draft → issued → partial/paid; issued records immutable. Changes use explicit credit notes/adjustments.
Traceability	Calculation inputs, formula version, results, evidence references and invoice history retained.

6. Discrepancy handling

Automatic: Routine, unambiguous imports require no lawyer approval. Warning: Missing or inconsistent facts are visible in the issues queue. Critical: Conflicts capable of changing financial or legal attribution block affected invoice issuance pending resolution. A passed validation is not a legal determination.

Storage, security and deployment

7. Local data layout

The installer deploys the app. On first launch, it creates an OS-standard user data directory. Imported files are copied there; the originals remain at their initial locations. SQLite stores relative managed-file IDs, original filenames, hashes and row/page provenance.

Storage	Example content
database/	claims.sqlite; transactional schema migrations and indexes.
documents/	Unmodified imported Excel/CSV/PDF copies, named by internal identifier.
exports/	Generated invoices and reports.
backups/	Encrypted backup packages including both database and source documents.
logs/	Operational events without sensitive full document content.

Important: Keeping files locally does not automatically encrypt them. Backup encryption, OS file permissions and sensitive-key handling require explicit implementation. Backups should also be exportable outside the app-data directory.

8. Cloud AI privacy boundary

Only explicitly authorized document content is sent to the selected cloud provider. Use HTTPS, minimal necessary text, secure credentials, redaction where practical, request logging without sensitive payloads, and documented provider retention/configuration. If consent or network access is missing, AI interpretation pauses; local import, structured data access and billing remain available.

9. Desktop security + packaging

Area	Implementation requirement
Renderer isolation	Electron contextIsolation on; sandbox where compatible; Node integration off; restrictive content security policy.
IPC	Preload exposes allowlisted typed methods only; main process validates every input and file path.
Python process	Packaged standalone backend executable, not a user-managed Python installation; private local IPC with authentication or OS-level restriction.
API credentials	Never embed shared production provider secrets in React or the packaged binary; use OS-protected per-user credentials or a properly secured service.
Packaging	Signed/notarized macOS and signed Windows builds; test clean installs, upgrades and uninstall behavior.
Recovery	Atomic file imports, database transactions, safe backup/restore of database and documents.

10. Release checks

Import Excel/CSV/PDF without changing originals; duplicate import does not double-count; file-to-row/page evidence opens correctly; critical conflicts block affected billing; calculations and rounding match test fixtures; issued invoices cannot be silently modified; AI content cannot be sent without authorization; backup/restore preserves records and source files; clean macOS and Windows installations run end-to-end.

Required client inputs before fee logic is final: Fee-agreement examples, invoice recipient rules, eligible claim base, tax/currency requirements and sample legal assignment cases.